

Experiment 'p_4_9_reverse_string' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_9_reverse_string

Natural language statement: Write a method that reads a word and returns the word in reverse.

Method signature: p_4_9_reverse_string(s: string) returns (reversed: string)

Ensures

- `|reversed| == |s|`
- forall `i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that reads a word and returns the word in reverse.

The signature should be:

`p_4_9_reverse_string(s: string) returns (reversed: string)`

The method should respect the following contract:

`ensures |reversed| == |s|, ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]`

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmfemd5ur1otB0QdwOSGV0lfnRWhl

Model: gpt-5.5-2026-04-23

Created at: 1780492244

Finish reason: stop

Usage: 318 tokens in, and 6111 tokens out

```
{
  reversed := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == s[|s| - 1 - j]
  {
    ghost var oldI := i;
    ghost var oldReversed := reversed;
    assert oldI < |s|;
    assert |oldReversed| == oldI;
    assert forall j :: 0 <= j < oldI ==> oldReversed[j] == s[|s| - 1 - j];

    reversed := reversed + s[|s| - 1 - i];
    assert reversed == oldReversed + s[|s| - 1 - oldI];
    i := i + 1;
    assert i == oldI + 1;
    assert |reversed| == i;
  }
}
```

```

forall j | 0 <= j < i
  ensures reversed[j] == s[|s| - 1 - j]
{
  if j < oldI {
    assert oldReversed[j] == s[|s| - 1 - j];
    assert reversed[j] == oldReversed[j];
  } else {
    assert j == oldI;
    assert reversed[j] == s[|s| - 1 - oldI];
    assert |s| - 1 - oldI == |s| - 1 - j;
  }
}
}
assert i == |s|;
assert forall j :: 0 <= j < |s| ==> reversed[j] == s[|s| - 1 - j];
}

```

Final Program

```

method p_4_9_reverse_string(s: string) returns (reversed: string)
  ensures |reversed| == |s|
  ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]
{
  reversed := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == s[|s| - 1 - j]
  {
    ghost var oldI := i;
    ghost var oldReversed := reversed;
    assert oldI < |s|;
    assert |oldReversed| == oldI;
    assert forall j :: 0 <= j < oldI ==> oldReversed[j] == s[|s| - 1 - j];

    reversed := reversed + [s[|s| - 1 - i]];
    assert reversed == oldReversed + [s[|s| - 1 - oldI]];
    i := i + 1;
    assert i == oldI + 1;
    assert |reversed| == i;

    forall j | 0 <= j < i
      ensures reversed[j] == s[|s| - 1 - j]
    {
      if j < oldI {
        assert oldReversed[j] == s[|s| - 1 - j];
        assert reversed[j] == oldReversed[j];
      } else {
        assert j == oldI;

```

```
        assert reversed[j] == s[|s| - 1 - oldI];
        assert |s| - 1 - oldI == |s| - 1 - j;
    }
}
}
assert i == |s|;
assert forall j :: 0 <= j < |s| ==> reversed[j] == s[|s| - 1 - j];
}
```

Total Token Usage

Input tokens: 318

Output tokens: 6111

Reasoning tokens: 5631

Sum of ‘total tokens’: 6429

Experiment Timings

Overall Experiment started at 1780492244669, ended at 1780492356131, lasting 111462ms (111.46 seconds)

Iteration #1 started at 1780492244672, ended at 1780492356131, lasting 111459ms (111.46 seconds)